

Ước lượng pose theo thời gian (Temporal Pose Estimation)

PHỤ THUỘC A/05-lan-truyen-sai-so-pixel-sang-pose (mục 8, phân loại nhiễu và hệ thống — chương này chỉ xử lý được loại thứ nhất); B/08-corner-refinement (hiệp phương sai của từng góc — đầu vào cho mọi phép hợp nhất có trọng số); B/10-xu-ly-ambiguity (điều kiện tiên quyết: phải chọn nghiệm tin cậy *trước* khi làm mượt, nếu không bạn đang làm mượt giữa hai nghiệm khác nhau).

TÓM TẮT

Chương này bàn cách dùng *thời gian* để giảm sai số, và nó có một điều kiện tiên quyết nghiêm ngặt phải nói ngay: mọi kỹ thuật ở đây chỉ hợp lệ sau khi bài toán hai nghiệm đã được xử lý — làm mượt giữa hai nghiệm khác nhau cho ra một pose không phải nghiệm nào cả (A/04 mục 2). Với điều kiện ấy được thoả, có bốn công cụ theo thứ tự phức tạp tăng dần: trung bình khi đứng yên, trung bình đúng cách trên đa tạp, lọc đệ quy với phép đo là *toạ độ góc* chứ không phải pose, và hợp nhất trên đồ thị nhân tử. Hai điểm kỹ thuật quan trọng nhất: *trung bình quaternion thô là sai* vì q và $-q$ biểu diễn cùng một xoay; và *phép đo phải là reprojection, không phải pose*, vì nhiễu là Gauss trên pixel chứ không Gauss trên $SE(3)$. Nếu chỉ nhớ một điều: *chương này chỉ chạm tới nguồn ngẫu nhiên* — nên nó trở nên quan trọng *hơn* sau khi B/13 đã triệt tiêu các nguồn hệ thống, chứ không phải thay thế cho việc đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đa tạp (manifold)** — không gian cong mà cục bộ trông như không gian Euclid. $SO(3)$ và $SE(3)$ là đa tạp: không có phép cộng, nên “trung bình” phải được định nghĩa lại.
- **trung bình Karcher** — điểm cực tiểu hoá tổng bình phương *khoảng cách trên đa tạp* tới các điểm cần trung bình. Với $SO(3)$ nó được tính bằng một vòng lặp: đưa mọi điểm về không gian tiếp tuyến tại ước lượng hiện tại, trung bình ở đó, cập nhật, lặp [1].
- **không gian tiếp tuyến** — xấp xỉ tuyến tính của đa tạp tại một điểm. Với $SO(3)$ tại một xoay R , nó là không gian ba chiều của các vectơ xoay nhỏ, và ánh xạ mũ đưa nó trở lại đa tạp.
- **lọc đệ quy** — cập nhật ước lượng theo từng phép đo mới thay vì giải lại toàn bộ. Kalman mở rộng (EKF) là dạng phổ biến nhất.
- **fixed-lag smoothing** — giữ một cửa sổ vài giây gần nhất và tối ưu lại toàn bộ cửa sổ mỗi khi có phép đo mới. Chính xác hơn lọc đệ quy vì nó tuyến tính hoá lại, tốn hơn vì nó giải một bài toán lớn hơn.
- **đổi mới (innovation)** — hiệu giữa phép đo thực tế và phép đo dự đoán từ trạng thái hiện tại. Độ lớn chuẩn hoá của nó là công cụ loại ngoại lai có cơ sở thống kê.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có bốn pose từ bốn khung liên tiếp và muốn lấy trung bình. Nếu bạn cộng bốn quaternion rồi chuẩn hoá, kết quả có thể sai hoàn toàn — không phải sai một chút. Cơ chế gì, và vì sao nó chỉ xuất hiện đôi khi?
- Q2. Một EKF chuẩn cho docking nhận đầu vào là pose từ PnP. Có một cách thiết kế tốt hơn, và nó đổi *đầu vào* chứ không đổi bộ lọc. Cách ấy là gì, và nó sửa vấn đề gì?

- Q3.** Robot đang dừng và bạn lấy trung bình 100 khung. Nêu chính xác dòng nào trong bảng ngân sách của C/15 được cải thiện, và dòng nào hoàn toàn không đổi.
- Q4.** Chương B/13 lập luận rằng teach-by-showing triệt tiêu sai số hệ thống. Vì sao điều đó làm chương này *quan trọng hơn* chứ không phải ít quan trọng đi?
- Q5.** Bạn muốn làm mượt pose để nó bớt nhảy. Nêu điều kiện tiên quyết mà nếu không thoả thì việc làm mượt biến một vấn đề nhìn thấy được thành một vấn đề ẩn.

1.1 Đặt vấn đề

Mọi chương trước đều xử lý *một* khung hình: một ảnh vào, một pose ra. Chương này hỏi: nhiều khung hình cho ta thêm gì?

Câu trả lời ngắn gọn là $1/\sqrt{N}$ — đã nêu ở B/08 mục 5. Nhưng câu trả lời ngắn gọn ấy giấu ba câu hỏi con mà chương này phải trả lời.

Câu hỏi thứ nhất: trung bình cái gì? Trung bình pose thì pose sống trên đa tạp và phép cộng không tồn tại. Trung bình tọa độ góc thì đơn giản hơn nhưng chỉ dùng được khi camera đứng yên. Hai lựa chọn ấy không tương đương, và mục 3 bàn.

Câu hỏi thứ hai: trung bình thế nào khi camera đang chuyển động? Lúc ấy pose thật đang đổi, nên trung bình thô là sai. Cần một mô hình chuyển động, và ta bước vào địa hạt của lọc và làm mượt — mục 4 và 5.

Câu hỏi thứ ba, và nó là câu hỏi quan trọng nhất: trung bình giúp được gì và không giúp được gì? Đây là chỗ A/05 mục 8 quay lại: trung bình giảm nguồn *ngẫu nhiên* theo $1/\sqrt{N}$ và hoàn toàn không chạm tới nguồn *hệ thống*. Nếu ngân sách của bạn bị chi phối bởi sai số constellation hoặc bởi méo còn sót, thì cả chương này gần như vô ích.

Sự thật cuối cùng ấy dẫn tới một quan hệ đáng chú ý với B/13, và nó trả lời Q4. Teach-by-showing triệt tiêu các nguồn hệ thống. Sau khi nó đã làm việc ấy, phần còn lại của ngân sách *chỉ gồm nguồn ngẫu nhiên* — tức chính xác thứ mà chương này xử lý. Nói cách khác: hai chương không cạnh tranh mà nối tiếp, và chương này trở nên quan trọng *hơn* sau khi chương kia đã làm việc của nó.

Có một điều kiện tiên quyết phải nói ngay, và nó là câu trả lời cho Q5. Mọi kỹ thuật trong chương giả định rằng các pose được đưa vào *cùng thuộc về một nghiệm*. Nếu bài toán hai nghiệm chưa được xử lý (B/10), thì bạn đang trung bình giữa hai giá trị cách nhau 2α , và kết quả rơi vào giữa — một pose không phải nghiệm nào cả, và theo A/04 mục 2 thì nó luôn rơi đúng vào tư thế fronto-parallel. Tệ hơn, việc làm mượt *xóa mất triệu chứng nhảy*, nên bạn mất dấu hiệu duy nhất cho biết mình có vấn đề. Đây là ví dụ điển hình của việc một kỹ thuật hợp lệ áp dụng sai thứ tự biến một vấn đề nhìn thấy được thành một vấn đề ẩn.

TRỰC GIÁC

Hình dung việc đo chiều cao một cái cây bằng cách nhìn từ xa, mỗi lần nhìn cho một con số hơi khác.

Nếu cây đứng yên và bạn đứng yên, trung bình nhiều lần nhìn là câu trả lời đúng — và càng nhiều lần càng tốt, theo căn bậc hai.

Nếu cây đang mọc, trung bình các lần nhìn cho bạn chiều cao *trung bình trong khoảng thời gian đo*, không phải chiều cao hiện tại. Muốn biết hiện tại thì phải có một mô hình về việc nó mọc nhanh thế nào, và dùng mô hình ấy để quy các phép đo cũ về thời điểm hiện tại. Đó là lọc.

Bây giờ đến chỗ tinh tế, và nó là chỗ mà pose khác chiều cao. Chiều cao là một con số — cộng hai chiều cao rồi chia đôi thì ra một chiều cao hợp lệ. Nhưng hướng thì không: cộng hai hướng “quay 170 độ” và “quay -170 độ” rồi chia đôi thì ra “quay 0 độ”, trong khi hai hướng ấy thật ra chỉ cách nhau 20 độ và trung bình đúng phải là 180 độ. Phép cộng thông thường không hiểu rằng hướng quay vòng lại.

Đó là lý do phải có một cách trung bình riêng cho hướng, và cách ấy là: chuyển mọi hướng về “lệch bao nhiêu so với một hướng tham chiếu” — lúc này chúng là những số nhỏ và cộng được — rồi trung bình, rồi cộng ngược trở lại. Lặp vài lần cho tới khi ổn định.

Và một chuyện cuối, về việc *đo cái gì*. Nếu bạn đo chiều cao cây bằng cách đo góc nhìn rồi tính ra chiều cao, thì sai số của bạn thật ra nằm ở phép *đo góc*, không phải ở chiều cao. Trung bình các chiều cao đã tính không giống với trung bình các góc rồi mới tính — và cách thứ hai đúng hơn, vì nó xử lý sai số ở đúng chỗ sai số sinh ra.

1.2 Trường hợp đơn giản nhất: camera đứng yên

Trường hợp này đáng có mục riêng vì với chiến lược stop-and-go của B/13 mục 4, nó là trường hợp *quan trọng nhất* chứ không phải trường hợp tầm thường.

Khi camera và tag đều đứng yên, pose thật là hằng số và bài toán trở thành ước lượng một hằng số từ nhiều phép đo nhiễu. Có hai cách, và chúng không tương đương.

Cách một, trung bình toạ độ góc rồi chạy PnP một lần. Lấy trung bình u_i qua N khung cho từng góc, rồi đưa các toạ độ trung bình ấy vào PnP.

Cách hai, chạy PnP mỗi khung rồi trung bình N pose. Đòi trung bình trên đa tạp (mục 3).

Cách một tốt hơn, và lý do đáng hiểu: nó xử lý sai số ở *đúng chỗ sai số sinh ra*. Nhiều là Gauss trên toạ độ pixel; trung bình N mẫu Gauss cho một mẫu Gauss với phương sai chia N , và PnP chạy trên dữ liệu ít nhiễu hơn là một bài toán tốt hơn. Cách hai chạy PnP trên dữ liệu nhiễu nhiễu rồi trung bình các kết quả — và vì PnP là phi tuyến, trung bình của các nghiệm không bằng nghiệm của bài toán trung bình.

Cách một cũng đơn giản hơn về cài đặt: không cần đa tạp, không cần Karcher, chỉ là phép cộng số thực. Và nó cho một lợi ích phụ: độ lệch chuẩn của N mẫu cho từng góc là một ước lượng *thực nghiệm* của σ , đúng thứ mà B/08 mini project cần và đúng thứ mà A/03 mục 6 cần làm trọng số.

Một chi tiết cài đặt: phải khớp đúng góc nào với góc nào giữa các khung. Với marker có ID và có hướng xác định thì việc này hiển nhiên; đừng giả định thứ tự trả về của detector là ổn định.

CẦN NHỚ

Với stop-and-go, cách đúng là: *trung bình toạ độ góc, rồi chạy PnP một lần.*

Không phải: chạy PnP mỗi khung rồi trung bình pose.

Ba lý do: nó xử lý nhiễu ở đúng chỗ nhiễu sinh ra; nó tránh hoàn toàn bài toán trung bình trên đa tạp; và nó cho σ thực nghiệm miễn phí như sản phẩm phụ.

Đây là một trong những chỗ mà cách đơn giản hơn cũng là cách đúng hơn, và nó đáng kiểm lại trong pipeline của bạn — nhiều cài đặt làm ngược vì thứ tự “phát hiện, tính pose, xử lý pose” nghe tự nhiên hơn.

1.3 Trung bình trên đa tạp: vì sao quaternion thô là sai

Mục này trả lời Q1, và nó cần thiết khi bạn *buộc* phải trung bình pose — chẳng hạn khi hợp nhất pose từ nhiều nguồn có bản chất khác nhau.

A. Cơ chế của lỗi

Một phép xoay biểu diễn bởi quaternion đơn vị q , và q cùng $-q$ biểu diễn *cùng một* phép xoay. Nếu bạn cộng bốn quaternion rồi chuẩn hoá, kết quả phụ thuộc vào dấu bạn tình cờ gán cho từng cái.

Ví dụ cụ thể: hai xoay gần giống nhau, một biểu diễn bởi q_1 và một bởi $q_2 \approx q_1$. Trung bình của q_1 và q_2 cho $\approx q_1$ — đúng. Nhưng nếu thư viện của bạn trả về $-q_2$ thay vì q_2 (và nó có quyền, vì hai cái tương đương), thì $q_1 + (-q_2) \approx 0$, và chuẩn hoá một vectơ gần không cho ra một hướng *hoàn toàn ngẫu nhiên*.

Đây là câu trả lời cho phần thứ hai của Q1: lỗi chỉ xuất hiện *đôi khi*, vì nó phụ thuộc vào việc thư viện tình cờ trả về dấu nào. Một hệ thống có thể chạy đúng hàng tháng rồi đột nhiên cho ra một pose vô nghĩa — và loại lỗi không tái lập được ấy là loại tổn thời gian gỡ nhất.

Biện pháp vá nhanh: trước khi cộng, đảo dấu mọi quaternion sao cho chúng cùng nằm trong một nửa cầu (chẳng hạn ép $q^\top q_{\text{ref}} > 0$). Nó chữa được triệu chứng và nó đáng làm ngay. Nhưng nó không phải trung bình đúng, vì trung bình số học của quaternion vẫn không phải trung bình trên $SO(3)$.

B. Trung bình Karcher

Cách đúng: trung bình Karcher, tức điểm cực tiểu hoá tổng bình phương khoảng cách *trên đa tạp* [1]. Thuật toán là một vòng lặp ngắn:

1. Khởi tạo $\bar{R}$ bằng một trong các R_i .
2. Với mỗi i , tính $\omega_i = \log(\bar{R}^\top R_i)$ — đưa R_i về không gian tiếp tuyến tại $\bar{R}$.
3. Tính $\bar{\omega} = \frac{1}{N} \sum_i \omega_i$ — ở đây cộng được vì đã là vectơ.
4. Cập nhật $\bar{R} \leftarrow \bar{R} \exp(\bar{\omega})$.
5. Lặp từ bước 2 cho tới khi $\|\bar{\omega}\|$ đủ nhỏ.

Vòng lặp hội tụ trong vài bước với các xoay gần nhau — đúng trường hợp của ta. Tài liệu tham chiếu đầy đủ hơn về chủ đề này là [2].

Cần nói rõ điều kiện: trung bình Karcher xác định duy nhất khi các xoay nằm trong một lân cận đủ nhỏ. Với các pose từ các khung liên tiếp thì điều kiện thoả thoải mái. Với hai nghiệm của bài toán ambiguity cách nhau 2α — có thể hàng chục độ — thì nó vẫn tính ra một kết quả, và kết quả ấy vẫn vô nghĩa. Trung bình đúng cách trên đa tạp *không* sửa được việc bạn đang trung bình những thứ không nên trung bình.

Với phần tịnh tiến thì không có vấn đề gì: nó sống trong $\mathbb{R}^3$ và trung bình số học là đúng. Nhưng nếu muốn trọng số hoá thì phải dùng hiệp phương sai, và hiệp phương sai của tịnh tiến *tương quan* với hiệp phương sai của xoay — nên tách chúng ra để xử lý riêng là một xấp xỉ. Cách đúng hoàn toàn là xử lý cả sáu chiều cùng lúc trên $SE(3)$, và đó là điều mà các thư viện như [Sophus](#) hay [manif](#) cung cấp.

1.4 Lọc: phép đo phải là reprojection

Khi camera chuyển động, cần một mô hình chuyển động và một bộ lọc. Mục này trả lời Q2, và nó là điểm kỹ thuật quan trọng nhất của chương.

A. Thiết kế thông thường và chỗ nó sai

Thiết kế thông thường: chạy PnP mỗi khung để được pose, rồi đưa pose ấy vào EKF như một phép đo, với một hiệp phương sai 6×6 nào đó.

Vấn đề: *hiều không phải Gauss trên $SE(3)$* . Nhiều là Gauss trên tọa độ pixel, và phép biến đổi từ pixel sang pose là phi tuyến. Kết quả là phân bố của pose ước lượng không phải Gauss, đặc biệt là ở các hướng yếu (chiều sâu, và góc nghiêng khi gần fronto-parallel). Ép nó thành Gauss bằng cách gán một hiệp phương sai 6×6 là một xấp xỉ, và nó tệ nhất đúng ở những hướng mà ta quan tâm nhất.

Có một vấn đề thứ hai, tinh tế hơn: nếu PnP hội tụ về nghiệm sai ở một khung nào đó, thì phép đo đưa vào bộ lọc là một ngoại lai rất lớn, và một EKF không có cơ chế bảo vệ sẽ bị kéo đi đáng kể.

B. Thiết kế đúng

Giữ nguyên bộ lọc, đổi phép đo: dùng *tọa độ góc trên ảnh* làm phép đo thay vì pose.

Cụ thể: trạng thái của bộ lọc là pose (và có thể cả vận tốc). Hàm đo lấy trạng thái, chiếu các điểm 3D của constellation ra ảnh, và trả về tọa độ dự đoán. Đối mới là hiệu giữa tọa độ đo được và tọa độ dự đoán — một đại lượng trong không gian pixel, nơi nhiều *thật sự* là Gauss.

Ba lợi ích, và chúng đáng kể.

Một, mô hình nhiều đúng. Hiệp phương sai của phép đo là Σ_i của từng góc (B/08 mục 7), một đại lượng có ý nghĩa vật lý rõ ràng và đo được, thay vì một ma trận 6×6 phải đoán.

Hai, loại ngoại lai ở đúng mức. Cổng chi-square trên đối mới của *từng góc* loại được một góc xấu mà vẫn giữ ba góc còn lại của cùng tag — điều mà cổng ở mức pose không làm được.

Ba, bài toán ambiguity được xử lý tự nhiên. Vì bộ lọc có một dự đoán từ trạng thái trước, nó tự nhiên chọn nghiệm gần dự đoán — tức nó thực hiện bậc 3 của B/10 mà không cần logic riêng. Nhưng cảnh báo về khoá cứng ở B/10 mục 4 vẫn áp dụng nguyên vẹn, và phải có cơ chế phát hiện.

Cái giá: hàm đo phức tạp hơn và Jacobian của nó là Jacobian của PnP (A/03 mục 4C) — nhưng bạn đã phải tính nó rồi, nên chi phí thêm gần như bằng không.

Nguyên tắc chung đằng sau chuyện này áp dụng rộng hơn bài toán marker: *giữ phép đo ở dạng nguyên thủy nhất mà mô hình nhiều còn đúng*, thay vì xử lý trước rồi đưa kết quả đã xử lý vào bộ ước lượng. Đây là một nguyên tắc quen thuộc trong ước lượng trạng thái robot [3].

1.5 Fixed-lag smoothing và đồ thị nhân tử

Bậc cao nhất, và nó đáng nêu ngắn gọn.

Lọc đệ quy tuyến tính hoá một lần tại mỗi bước và không quay lại. Nếu tuyến tính hoá ở một điểm tồi — chẳng hạn ngay sau khi hệ thống mất tag rồi thấy lại — sai số ấy không được sửa.

Fixed-lag smoothing giữ một cửa sổ vài giây gần nhất và tối ưu lại toàn bộ cửa sổ mỗi khi có phép đo mới. Nó tuyến tính hoá lại ở mọi bước lặp, nên nó chính xác hơn, và nó xử lý được các phép

đo đến không theo thứ tự thời gian. Với bài toán docking — nơi đoạn quan trọng chỉ vài giây cuối — cửa sổ cố định là một lựa chọn rất tự nhiên: *giữ toàn bộ giai đoạn tiếp cận cuối trong cửa sổ*.

Về mặt cài đặt, cách hiện đại là biểu diễn bài toán dưới dạng *đồ thị nhân tử* [4] và dùng một thư viện như [GTSAM](#) hoặc [Ceres](#). Các nhân tử: reprojection cho mỗi góc mỗi khung, chuyển động (từ odometry bánh xe hoặc mô hình vận tốc), và có thể cả IMU. Ưu điểm so với EKF: nó là cùng một khung với bài toán hiệu chuẩn constellation ở B/09 mục 5, nên bạn dùng lại được công cụ; và nó cho phép đưa các ràng buộc thêm vào một cách tự nhiên — chẳng hạn ràng buộc sàn phẳng đã nêu ở B/10 mục 5.

Khi nào đáng dùng: khi bạn đã fuse nhiều nguồn (marker, odometry, IMU), hoặc khi bạn cần xử lý các phép đo trễ. Với một hệ chỉ có marker và dùng stop-and-go, một EKF đơn giản — hoặc thậm chí chỉ mục 2 — là đủ, và sự phức tạp thêm không trả lại gì.

1.6 Điều chương này không làm được

Mục ngắn nhưng quan trọng nhất về mặt định vị chương trong cây. Nó trả lời Q3 và Q4.

Chương này chỉ chạm tới nguồn ngẫu nhiên. Theo phân loại của A/05 mục 8: nhiễu định vị góc, nhiễu ảnh, và các dao động ngẫu nhiên khác. Chúng giảm theo $1/\sqrt{N}$.

Chương này không chạm tới nguồn hệ thống: méo còn sót, sai số intrinsic, sai số constellation, sai số hand-eye, bias tâm ellipse, rolling shutter và blur nếu còn chuyển động. Lấy trung bình một triệu khung thì chúng vẫn nguyên vẹn.

Đây là câu trả lời cho Q3, và nó đáng phát biểu dưới dạng một lời khuyên thực hành: *trước khi đầu tư vào bộ lọc phức tạp, hãy vẽ đồ thị σ_N theo N* (B/08 mục 5A). Nếu nó bão hoà sớm thì bạn đang bị giới hạn bởi thiên lệch, và một EKF tinh vi hơn không giúp gì — việc cần làm là đi tìm nguồn thiên lệch.

Và đây là câu trả lời cho Q4, và nó là quan hệ đáng chú ý nhất của chương. B/13 mục 3 triết tiêu các nguồn hệ thống bằng teach-by-showing. Sau khi nó làm việc ấy, phần còn lại của ngân sách *chỉ gồm nguồn ngẫu nhiên*. Nói cách khác: teach-by-showing làm cho chương này trở thành chương quyết định phần dư, chứ không phải làm nó thừa. Hai chương là một cặp: một cái xoá các dòng hệ thống, một cái chia các dòng ngẫu nhiên cho $\sqrt{N}$.

CẦN NHỚ

Thứ tự đúng của ba việc, và đảo thứ tự thì hỏng:

Một, chọn nghiệm tin cậy (B/10). Làm mượt trước khi làm việc này thì bạn đang trung bình giữa hai nghiệm và kết quả luôn rơi vào fronto-parallel.

Hai, xử lý các nguồn hệ thống (B/09 mục 5, B/12, hoặc B/13 mục 3). Làm mượt không chạm tới chúng, nên làm mượt trước thì bạn chỉ đang làm cho một con số sai trở nên ổn định.

Ba, rồi mới làm mượt — và lúc này nó thật sự hiệu quả, vì phần còn lại đúng là nguồn ngẫu nhiên.

Sai lầm phổ biến nhất là làm bước ba trước, vì nó dễ nhất và vì đầu ra trông đẹp ngay lập tức. Nhưng “trông đẹp” ở đây nghĩa là đã xoá mất các triệu chứng chẩn đoán.

1.7 Giới hạn và chế độ hỏng

Làm mượt xoá triệu chứng. Đã bàn ở mục 1 và 6. Đây là chế độ hỏng nghiêm trọng nhất của chương vì nó không gây ra lỗi mới mà *giấu* lỗi cũ.

Trung bình Karcher không sửa được việc trung bình sai thứ. Nó tính đúng trung bình của tập bạn đưa vào; nếu tập ấy gồm hai nghiệm khác nhau thì kết quả vẫn vô nghĩa.

$1/\sqrt{N}$ có điều kiện. Nhiều phải độc lập giữa các khung. B/08 mục 5A liệt kê ba cách vi phạm và cách phát hiện.

Lọc giả định mô hình chuyển động đúng. Nếu robot tăng tốc đột ngột và mô hình giả định vận tốc không đổi, bộ lọc sẽ trễ. Với docking tốc độ thấp thì ít quan trọng, nhưng nó là lý do nữa để thích stop-and-go.

Mọi thứ giả định đồng bộ thời gian đúng. Nếu nhãn thời gian của ảnh lệch so với nhãn thời gian của odometry, bộ lọc hợp nhất hai nguồn ở sai thời điểm và kết quả có thiên lệch phụ thuộc vận tốc — một triệu chứng giống hết rolling shutter (A/06 mục 2), và phân biệt hai nguyên nhân đòi phép thử riêng.

1.8 Thực hành

Bốn việc.

Một, với stop-and-go, dùng cách của mục 2: trung bình toạ độ góc rồi chạy PnP một lần. Đơn giản hơn và đúng hơn.

Hai, nếu phải trung bình pose, đảo dấu quaternion về cùng nửa cầu ngay lập tức như một biện pháp vá, rồi cài trung bình Karcher khi có thời gian.

Ba, nếu dùng bộ lọc, đổi phép đo sang toạ độ góc thay vì pose. Chi phí thêm gần bằng không vì Jacobian đã có.

Bốn, vẽ đồ thị σ_N theo N trước khi đầu tư vào bất kỳ bộ lọc phức tạp nào.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Điều kiện tiên quyết của cả chương	Nghiem phải được chọn tin cậy <i>trước</i> (B/10). Làm mượt giữa hai nghiệm cho pose rơi vào fronto-parallel (A/04 mục 2) và xoá mất triệu chứng nhảy — biến vấn đề nhìn thấy được thành vấn đề ẩn.
••• Với camera đứng yên: cách đúng	Trung bình toạ độ góc rồi chạy PnP một lần. Không phải: PnP mỗi khung rồi trung bình pose. Ba lý do: xử lý nhiễu ở đúng chỗ nó sinh ra; tránh bài toán đa tạp; và cho σ thực nghiệm miễn phí.
••• Vì sao trung bình quaternion thô sai	q và $-q$ biểu diễn cùng một xoay, nên tổng phụ thuộc dấu thư viện tình cờ trả về. Hai xoay gần nhau với dấu ngược cho tổng gần không, và chuẩn hoá vectơ gần không ra hướng <i>ngẫu nhiên</i> .
••• Vì sao lỗi ấy chỉ xuất hiện đôi khi	Vì nó phụ thuộc dấu mà thư viện tình cờ trả về. Hệ có thể chạy đúng hàng tháng rồi đột nhiên cho một pose vô nghĩa — loại lỗi không tái lập được, tốn thời gian gỡ nhất.
•• Vá nhanh và cách đúng	Vá: đảo dấu mọi quaternion về cùng nửa cầu ($q^\top q_{\text{ref}} > 0$). Đúng: trung bình Karcher — lập log, trung bình trong không gian tiếp tuyến, exp, cập nhật [1].

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Điểm kỹ thuật quan trọng nhất của chương	Trong bộ lọc, phép đo phải là <i>toạ độ góc</i> chứ không phải pose. Vì nhiều là Gauss trên <i>pixel</i> , không Gauss trên $SE(3)$; ép nó thành Gauss trên pose là xấp xỉ tệ nhất đúng ở các hướng yếu.
●●● Ba lợi ích của phép đo là reprojection	Mô hình nhiễu đúng (Σ_i đo được thay vì ma trận 6×6 phải đoán); loại ngoại lai ở mức <i>từng góc</i> thay vì cả pose; và chọn nghiệm ambiguity tự nhiên qua dự đoán.
●● Fixed-lag smoothing	Giữ cửa sổ vài giây và tối ưu lại toàn bộ mỗi bước — tuyến tính hoá lại nên chính xác hơn lọc đệ quy, và xử lý được phép đo đến trễ. Với docking, chọn cửa sổ <i>chứa toàn bộ pha tiếp cận cuối</i> .
●● Khi nào dùng đồ thị nhân tử	Khi fuse nhiều nguồn (marker + odom + IMU) hoặc cần xử lý đo trễ. Ưu điểm phụ: cùng khung với hiệu chuẩn constellation ở B/09 mục 5 [4]. Với hệ chỉ marker + stop-and-go thì EKF đơn giản là đủ.
●●● Chương này KHÔNG làm được gì	Không chạm tới nguồn <i>hệ thống</i> : méo còn sót, intrinsic, constellation, hand-eye, bias ellipse, rolling shutter/blur khi còn chuyển động. Trung bình một triệu khung thì chúng vẫn nguyên vẹn.
●●● Quan hệ với B/13	Không cạnh tranh mà <i>nối tiếp</i> . Teach-by-showing xoá các dòng hệ thống; chương này chia các dòng ngẫu nhiên cho $\sqrt{N}$. Nên chương này quan trọng <i>hơn</i> sau khi chương kia đã làm việc của nó.
●●● Thứ tự ba việc, đảo thì hồng	Chọn nghiệm → xử lý nguồn hệ thống → rồi mới làm mượt. Sai lầm phổ biến là làm bước ba trước vì nó dễ nhất và đầu ra trông đẹp ngay — nhưng “đẹp” ở đây nghĩa là đã xoá triệu chứng chẩn đoán.
●● Chế độ hồng do lệch đồng bộ thời gian	Nhân thời gian ảnh lệch so với odometry ⇒ hợp nhất ở sai thời điểm ⇒ thiên lệch phụ thuộc vận tốc — triệu chứng <i>giống hệt</i> rolling shutter (A/06 mục 2). Cần phép thử riêng để phân biệt.
● Nguyên tắc chung	Giữ phép đo ở dạng <i>nguyên thủy nhất</i> mà mô hình nhiễu còn đúng, thay vì xử lý trước rồi đưa kết quả đã xử lý vào bộ ước lượng [3].

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Sophus , manif [5]	Cài $SO(3)$, $SE(3)$, log/exp, và trung bình trên đa tạp	Đừng tự cài Karcher — dùng thư viện
GTSAM [4]	Đồ thị nhân tử với fixed-lag smoother sẵn có	Cùng công cụ với B/09 mục 5 — dùng lại được
Ceres [6]	Tối ưu cửa sổ trượt với tham số hoá trên đa tạp	Lựa chọn khi đã dùng Ceres cho BA constellation
Rotation averaging [2]	Tài liệu tham chiếu đầy đủ cho mục 3	Đọc §3 nếu cần trung bình có trọng số hoặc bền với ngoại lai
tagslam [7]	Fuse marker với odometry trong factor graph	Ví dụ kiến trúc cho mục 5; mức tin C

MINI PROJECT · So bốn cách hợp nhất theo thời gian trên cùng dữ liệu · 2 ngày

Mục tiêu biết chính xác mỗi mức phức tạp thêm vào trả lại bao nhiêu, để không đầu tư vào một EKF khi một phép trung bình đã đủ.

Dữ liệu	một chuỗi ảnh thật của quỹ đạo tiếp cận, ghi lại kèm odometry bánh xe và — nếu có — một chuẩn tham chiếu ở điểm cuối.
Các bước	cài bốn đường: (A) PnP một khung, không hợp nhất; (B) trung bình toạ độ góc qua N khung rồi PnP (chỉ ở các đoạn dừng); (C) EKF với phép đo là <i>pose</i> ; (D) EKF với phép đo là <i>toạ độ góc</i> ; chạy cả bốn trên cùng chuỗi; so độ tản mát của pose ở điểm cuối qua nhiều lần chạy, và so sai lệch so với chuẩn tham chiếu nếu có; đồng thời ghi lại chi phí tính toán của từng đường.
Xong khi	bạn có một bảng bốn hàng với ba cột: độ tản mát, sai lệch so với chuẩn, chi phí. Dự đoán của chương: B thắng A rõ rệt về tản mát; D thắng C ở các đoạn mà ambiguity nặng hoặc có góc bị che; và <i>sai lệch so với chuẩn</i> của cả bốn gần như nhau — vì đó là thành phần hệ thống mà không đường nào chạm tới. Kiểm tra chéo bắt buộc: nếu cột sai lệch của bốn đường <i>khác nhau</i> đáng kể thì có gì đó không đúng — hoặc bạn đang so ở các thời điểm khác nhau, hoặc một trong các đường đang chọn nghiệm ambiguity khác.
Nối với	C/15 nhận kết quả cột ba để biết nên đầu tư ở đâu; B/13 mục 4 nhận N tối ưu cho stop-and-go.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Cơ chế: q và $-q$ biểu diễn *cùng một* phép xoay, nên tổng của một tập quaternion phụ thuộc vào dấu mà bạn tình cờ gán cho từng cái. Nếu hai xoay gần giống nhau nhưng thư viện trả về q_1 và $-q_2$ với $q_2 \approx q_1$, thì tổng $q_1 + (-q_2) \approx 0$, và chuẩn hoá một vectơ gần không cho ra một hướng hoàn toàn ngẫu nhiên — sai hẳn, không phải sai một chút. Nó chỉ xuất hiện đôi khi vì nó phụ thuộc vào dấu mà thư viện tình cờ trả về, thứ có thể đổi theo đường đi của thuật toán hoặc theo dữ liệu. Hệ quả thực tế: một hệ thống có thể chạy đúng hàng tháng rồi đột nhiên cho một pose vô nghĩa, và loại lỗi không tái lập được ấy là loại tổn thời gian gở nhất. Vá nhanh: ép mọi quaternion về cùng nửa cầu trước khi cộng. Cách đúng: trung bình Karcher trên $SO(3)$ [1].
- A2. Đổi phép đo từ *pose* sang *toạ độ góc trên ảnh*: hàm đo của bộ lọc lấy trạng thái pose, chiếu các điểm 3D của constellation ra ảnh, và đổi mới là hiệu giữa toạ độ đo được và toạ độ dự đoán. Nó sửa vấn đề mô hình nhiều: nhiều là Gauss trên *pixel*, không Gauss trên $SE(3)$, và phép biến đổi giữa hai không gian là phi tuyến — nên ép phân bố pose thành Gauss bằng một ma trận 6×6 là một xấp xỉ, và nó tệ nhất đúng ở các hướng yếu (chiều sâu, góc nghiêng gần fronto-parallel), tức đúng những hướng ta quan tâm nhất. Ba lợi ích kèm theo: hiệp phương sai đầu vào là Σ_i của từng góc — đại lượng đo được thay vì phải đoán; cổng chi-square loại được *một góc* xấu mà giữ ba góc còn lại; và bộ lọc tự nhiên chọn nghiệm ambiguity gần dự đoán. Chi phí thêm gần bằng không vì Jacobian đã phải tính cho PnP.
- A3. Cải thiện: mọi dòng thuộc loại *ngẫu nhiên* — chủ yếu là dòng “nhiều định vị góc”, giảm theo $1/\sqrt{100} = 10$ lần. Hoàn toàn không đổi: mọi dòng thuộc loại *hệ thống* — méo còn sót, sai số intrinsic, sai số constellation so với mô hình, sai số hand-eye, bias tâm ellipse nếu dùng marker tròn. Chúng nguyên vẹn sau một trăm khung cũng như sau một triệu khung. Hệ quả thực hành trực tiếp: trước khi đầu tư vào bộ lọc phức tạp hơn, hãy vẽ đồ thị σ_N theo N trên trục log-log (B/08 mục 5A); nếu nó bão hoà sớm thì bạn đang bị giới hạn bởi thiên lệch và một EKF tinh vi hơn không giúp gì — việc cần làm là đi tìm nguồn thiên lệch.
- A4. Vì hai chương xử lý hai loại nguồn khác nhau và chúng nối tiếp chứ không cạnh tranh. Teach-by-showing triệt tiêu các nguồn *hệ thống* — intrinsic sai, constellation lệch, hand-eye sai — bằng cách làm chúng xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và trừ đi nhau. Sau khi nó làm việc ấy, phần còn lại của ngân sách *chỉ gồm nguồn ngẫu nhiên*, tức chính xác thứ mà chương này chia cho $\sqrt{N}$. Nói cách khác, trước khi có teach-by-showing thì chương này chỉ cải thiện một phần nhỏ của ngân sách và bị các dòng hệ thống át đi; sau khi có nó thì chương này quyết định toàn bộ phần dư. Đây là một ví dụ của việc thứ tự làm quyết định giá trị của từng bước: cùng một kỹ thuật, giá trị khác nhau tùy vào việc gì đã được làm trước.
- A5. Điều kiện tiên quyết: bài toán hai nghiệm phải được xử lý trước (B/10). Nếu không, bạn đang làm mọt giữa hai giá trị cách nhau 2α , và theo A/04 mục 2 thì trung bình của chúng luôn rơi đúng vào tư thế fronto-parallel — một pose không phải nghiệm nào cả và sai một cách có hệ thống. Nhưng tác hại lớn hơn nằm ở chỗ khác: việc làm mọt *xoá mất triệu chứng nhấy*, tức xoá

mất dấu hiệu duy nhất cho biết bạn đang có vấn đề. Trước khi làm mượt, bạn có một hệ thống nhảy loạn — khó chịu nhưng trung thực, và nó đang báo cho bạn biết điều gì đó sai. Sau khi làm mượt, bạn có một hệ thống êm ái và sai có hệ thống, không có tín hiệu nào. Đây là ví dụ điển hình của việc một kỹ thuật hợp lệ áp dụng sai thứ tự biến một vấn đề nhìn thấy được thành một vấn đề ẩn.

NÓI LẠI THẬT ĐƠN GIẢN

Chụp một tấm ảnh thì có nhiều; chụp nhiều tấm rồi lấy trung bình thì bớt nhiều. Đó là toàn bộ ý tưởng của chương này, và phần còn lại là chi tiết về việc làm sao trung bình cho đúng.

Chi tiết thứ nhất: khi robot đứng yên, đừng tính vị trí từng tấm rồi trung bình các vị trí. Hãy trung bình *toạ độ mấy cái góc* trước, rồi mới tính vị trí một lần. Nghe như chuyện thứ tự vụn vặt nhưng nó quan trọng, vì nhiều nằm ở mấy cái góc chứ không nằm ở vị trí — xử lý nhiều ở đúng chỗ nó sinh ra thì tốt hơn.

Chi tiết thứ hai: hướng quay thì không cộng được như số thường. Lấy trung bình của “quay 170 độ” và “quay âm 170 độ” theo kiểu cộng chia đôi thì ra 0 độ, trong khi câu trả lời đúng phải là 180. Với cách máy tính lưu hướng quay thì chuyện này còn tệ hơn: có hai cách viết cho cùng một hướng, và nếu chẳng may lấy mỗi cái một kiểu rồi cộng lại thì ra một kết quả hoàn toàn vô nghĩa. Đáng sợ ở chỗ nó chỉ xảy ra thỉnh thoảng, nên hệ thống chạy đúng hàng tháng rồi bỗng dưng loạn một lần. Chi tiết thứ ba, và là chi tiết đáng nhớ nhất: khi robot đang chạy và cần một bộ lọc, đừng đưa *vị trí đã tính* vào bộ lọc. Hãy đưa *toạ độ mấy cái góc trên ảnh* vào. Lý do vẫn là lý do cũ: cái bị nhiễu là mấy cái góc, còn vị trí chỉ là thứ suy ra từ chúng. Bộ lọc làm việc với thứ bị nhiễu thật thì nó hiểu đúng mình đang đối phó với cái gì.

Và bây giờ là điều quan trọng nhất, thứ mà cả chương này chỉ là một nửa của nó. Chụp nhiều tấm chưa được cái *run tay* nhưng không chưa được cái *lệch ngắm*. Nếu tấm marker của bạn lệch một milimét so với bản vẽ, thì một triệu tấm ảnh vẫn lệch đúng một milimét ấy.

Cho nên chương này chỉ có giá trị sau khi đã dọn xong mấy cái lệch ngắm — đo lại tấm marker, hiệu chuẩn tứ tế, hoặc dùng mẹo chụp ảnh mẫu ở chương sau. Làm ngược thứ tự thì bạn chỉ đang làm cho một con số sai trở nên ổn định hơn, và cái ổn định ấy lại che mất dấu hiệu duy nhất cho biết có gì đó không đúng.

Chỗ không rút gọn được: ranh giới giữa “run tay” và “lệch ngắm” không nhìn ra được từ một lần đo, nhưng có một biểu đồ rõ ràng cho biết: vẽ xem sai số giảm thế nào khi tăng số tấm chụp. Giảm đều thì là run tay, cứ chụp thêm. Giảm rồi dừng ở một mức nào đó thì là lệch ngắm, và chụp thêm vô ích.

Một câu: Trung bình đúng chỗ, đúng cách, và đúng thứ tự — và nhớ rằng nó chỉ chữa được một nửa vấn đề.

1.9 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *trung bình toạ độ góc rồi PnP một lần tốt hơn PnP mỗi khung rồi trung bình pose* — bị bác bỏ nếu mô phỏng cho thấy hai cách cho cùng độ tản mát. Tôi dẫn nó bằng lập luận về chỗ nhiễu sinh ra và tính phi tuyến của PnP (cửa a); nó chưa qua cửa (b). Chênh lệch có thể nhỏ khi nhiễu nhỏ — vì khi ấy PnP gần tuyến tính — nên đây là mệnh đề mà tôi dự đoán sẽ đúng về chiều nhưng khiêm tốn về độ lớn.

Mệnh đề thứ hai — *phép đo là reprojection tốt hơn phép đo là pose* — bị bác bỏ nếu đường C và đường D trong mini project cho kết quả tương đương ở mọi đoạn. Tôi dự đoán chênh lệch tập trung ở các đoạn có góc bị che hoặc ambiguity nặng, và nếu chuỗi dữ liệu của bạn không có đoạn nào như thế thì hai đường sẽ giống nhau — điều đó không bác bỏ mệnh đề, nó chỉ nói dữ liệu chưa đủ khắc nghiệt.

Mệnh đề thứ ba — *làm mượt không chạm tới nguồn hệ thống* — là một mệnh đề định nghĩa hơn là một mệnh đề thực nghiệm: nếu một nguồn giảm khi lấy trung bình thì theo định nghĩa nó là

ngẫu nhiên. Điều đáng kiểm là phân loại cụ thể: nguồn nào trên hệ của bạn thật sự là hệ thống. Cột “sai lệch so với chuẩn” trong mini project là phép kiểm ấy.

1.10 Kết nối

Chương này là chương kết nối nhiều nhất trong Phần B và nó nằm ở giữa một chuỗi phụ thuộc nghiêm ngặt.

Nối lên trên. A/05-lan-truyen-sai-so-pixel-sang-pose mục 8 cấp phân loại quyết định phạm vi của cả chương. A/03-pnp-va-uoc-luong-tu-the mục 4C cấp Jacobian mà mục 4B ở đây dùng làm hàm đo. A/04-pose-ambiguity-target-phang mục 2 cấp cảnh báo về trung bình giữa hai nghiệm.

Nối ngang, theo thứ tự phụ thuộc. B/10-xu-ly-ambiguity là điều kiện tiên quyết cứng: phải chạy trước. B/08-corner-refinement cấp Σ_i cho mục 4B, và mục 5A ở đó cấp đồ thị chẩn đoán mà mục 6 ở đây khuyến nghị. B/09-multi-marker-constellation mục 5 và B/12-calibration-cho-docking là nơi các nguồn hệ thống được xử lý — tức nơi phải làm xong trước khi chương này có giá trị đầy đủ. B/13-docking-control mục 3 là lựa chọn thay thế cho việc ấy, và mục 4 ở đó tạo điều kiện cho mục 2 ở đây.

Nối xuống. C/15-ngan-sach-sai-so nhận từ mục 6 một yêu cầu về cách trình bày bảng ngân sách: phải ghi rõ dòng nào chia được cho $\sqrt{N}$ và dòng nào không, vì hai loại ấy phản ứng khác nhau với chiến lược thời gian. C/16-danh-gia-va-nghiem-thu nhận mini project của chương này, và nó cũng nhận một cảnh báo phương pháp luận: khi đo độ tán mát của một hệ có làm mượt, con số thu được *không* phải độ chính xác — làm mượt giảm tán mát mà không giảm thiên lệch, nên nó làm con số trông đẹp hơn thực tế.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao làm mượt trước khi chọn nghiệm biến một vấn đề nhìn thấy được thành một vấn đề ẩn?
2. Với camera đứng yên, nêu hai cách trung bình và ba lý do cách thứ nhất tốt hơn.
3. Mô tả cơ chế làm trung bình quaternion thô cho kết quả ngẫu nhiên, và giải thích vì sao lỗi ấy khó tái lập.
4. Trong một bộ lọc, phép đo nên là gì và vì sao? Nêu ba lợi ích cụ thể của lựa chọn đúng.
5. Bạn lấy trung bình 100 khung. Phân loại các dòng ngân sách thành hai nhóm theo việc chúng có được cải thiện không, và nêu một dòng ở mỗi nhóm.
6. Vì sao B/13 làm chương này quan trọng hơn chứ không phải ít quan trọng đi?
7. Một hệ có lệch đồng bộ thời gian cho triệu chứng giống hết một hiệu ứng khác trong cây. Hiệu ứng nào, và vì sao cần một phép thử riêng để phân biệt?

Nguồn

- [1] M. Moakher, “Means and Averaging in the Group of Rotations,” *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 1, pp. 1–16, 2002, mức tin A; trung bình Karcher trên $SO(3)$, và vì sao trung bình quaternion thô là sai.
- [2] R. Hartley, J. Trunpf, Y. Dai, et al., “Rotation Averaging,” *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.
- [3] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; ước lượng trên đa tạp, trung bình trên $SO(3)$.
- [4] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [5] J. Solà, J. Deray, and D. Atchuthan, “A Micro Lie Theory for State Estimation in Robotics,” *arXiv:1812.01537*, 2018, mức tin B; tiền ấn nhưng được dùng rộng rãi làm tài liệu tham chiếu.
- [6] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [7] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.